SOC INVESTIGATION & MALWARE ANALYSIS REPORT

Technical Investigation Report: Malicious_AutoIT

Static Triage, PE Structure Inspection, and Embedded Script Analysis

1. Document Control & Author

Field	Details
Analyst / Author	Jonathan Di Genova (Junior SOC Analyst & Cybersecurity Specialist)
Date	November 12, 2026
Target Lab / Platform	Hack The Box — Malicious_AutoIT (Sherlock Lab, Malware Analysis, Very Easy)
Classification	Personal Study Document & Blue Team Portfolio Documentation

2. Case Overview & Scope

Our organization's Security Operations Center (SOC) detected suspicious endpoint activity associated with an unknown executable suspected of wrapping a compiled AutoIt script. The primary objective of this technical investigation was to perform safe static triage on the binary inside an isolated analysis workstation, inspect its Portable Executable (PE) metadata, extract the embedded AutoIt source code, and document the execution chain and Indicators of Compromise (IoCs) without relying on live execution.

> **Operational Confidentiality Note:** In strict compliance with platform policies and responsible portfolio reporting standards, raw cryptographic hashes, exact memory offsets, decoded IoC strings, and direct task answers are abstracted or omitted. All analytical phases are verified as completed.

3. Personal Investigation Timeline

Phase	Tool / Environment	Analytical Action & Problem Solving	Status
Phase 1 Static Profiling	Detect It Easy (DIE)	Computed cryptographic file fingerprint and evaluated overall file entropy to check for packing or compression layers.	**Task Completed**
Phase 2 PE Header Inspection	Detect It Easy (DIE)	Navigated the PE structure view to inspect section alignments, the `.text` section virtual address, the Original Entry Point (OEP), and the compilation time-date stamp.	**Task Completed**
Phase 3 Script Extraction	Windows CLI / Python (`AutoIt-Ripper`)	Resolved initial path and Python module resolution errors by navigating to the tool root directory and invoking `python -m autoit_ripper.cli` to unpack the embedded `.au3` source file.	**Task Completed**
Phase 4 Code Review & IoC Triage	Notepad / Static Code Review	Audited the extracted AutoIt script to identify remote staging URLs, convert hexadecimal strings into system directory paths, and trace Windows API memory-execution calls.	**Task Completed**

4. Key Evidence & Technical Artifacts

- **Binary & Entropy Profile (Tasks 1–2):** Static inspection confirmed a 32-bit Windows Portable Executable compiled with AutoIt. Elevated entropy measurements across the code section indicated packed or compressed payload data requiring specialized extraction rather than plain-text string searches.
- **PE Architectural Metadata (Tasks 3–5):** Reviewing the PE header and section table established the memory layout of the executable, including the relative virtual address of the `.text` code section, the entry point address, and the recorded compilation timestamp.
- **Command-and-Control & Staging Logic (Task 6):** Source code analysis of the unpacked `.au3` script revealed functions utilizing AutoIt's native `InetRead` primitive to fetch secondary text payloads from an external command-and-control domain, followed by string-replacement routines to reconstruct shellcode in memory.
- **Obfuscated System Path & API Execution Chain (Tasks 7–8):** The script utilized `BinaryToString` to decode a hexadecimal-encoded Windows system path at runtime. Finally, it allocated a byte buffer via `DllStructCreate` and leveraged `DllCall` against `user32.dll` (invoking `CallWindowProcW`) to execute the retrieved payload directly in memory, bypassing traditional disk-write detections.

5. Lessons Learned & Hardening Recommendations

Investigating this sample demonstrated why Junior SOC Analysts must combine PE structural triage with script-level decompilation. Adversaries frequently abuse legitimate automation engines like AutoIt to wrap malicious loaders, pairing simple obfuscation (hex encoding and string substitution) with Windows API callbacks to achieve stealthy in-memory execution.

- **Endpoint Detection & Behavioral Monitoring:** Configure EDR rules to alert on non-standard binaries invoking `CallWindowProcW` or allocating executable memory structures from script interpreters.
- **Application Control & Allowlisting:** Restrict the execution of unsigned or unapproved AutoIt-compiled executables (`.exe` wrappers) within user-writable directories such as Desktop, Downloads, or AppData.
- **Network Egress Filtering:** Enforce proxy inspection and block outbound HTTP requests initiated by anomalous workstation binaries to newly registered or uncategorized domains.